Software Cost Estimation with COCOMO, Effort Estimation, and Project Risk Management

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Explain the difference between effort, duration, cost, and size.

• Describe common software estimation approaches and their assumptions.

• Apply the basic COCOMO model using the correct equations and units.

• Identify project risks, assess probability and impact, and plan responses.

• Use a simple risk register to support a software project.

Running case study: The team is estimating the work to deliver a College Event Management System (CEMS) before the college festival.

1. Why Estimate Software Projects?

Software estimates help a team decide whether a proposed scope can be delivered with available people, time, and budget. They support planning, staffing, scheduling, bidding, and conversations about trade-offs. An estimate is a forecast based on available information—not a guarantee.

Estimation is difficult because requirements may be incomplete, complexity varies, dependencies can fail, and teams learn while building. A useful estimate states its assumptions, uncertainty, scope, and units.

Example: Saying “CEMS will take 30 days” is incomplete. Does that mean 30 calendar days or working days? Is it elapsed duration or 30 person-days of effort? Which features, team size, and testing activities are included?

2. Size, Effort, Duration, and Cost

Concept	Meaning	Example
Size	Amount of software to be built, estimated using a defined measure	Estimated 8 KLOC or a specified number of function points
Effort	Work performed by people, often measured in person-hours or person-months	A total of 12 person-months
Duration	Elapsed time from start to finish	A project lasting 4 calendar months
Cost	Money spent on people, tools, hosting, and other resources	Estimated labour cost plus infrastructure and other expenses

Effort and duration are not interchangeable. Four people working for one month do not always complete the same work as one person working for four months: communication, dependencies, onboarding, and task parallelism affect outcomes.

3. Common Estimation Approaches

Expert judgment

Experienced people estimate work using knowledge of similar tasks, technologies, and team conditions. It is useful when historical data is limited, but assumptions and uncertainty should be made explicit.

Estimation by analogy

Compare the proposed project or feature with a completed one and adjust for differences in scope, complexity, team experience, and constraints.

Decomposition / bottom-up estimation

Break scope into smaller work packages or activities, estimate each, and aggregate the estimates. It can expose overlooked work but depends on the quality of the breakdown.

Parametric estimation

Use a formula or model that relates size or other project characteristics to effort. COCOMO is a classic example. The model's output is only as useful as its inputs and assumptions.

Three-point estimation

Use optimistic (O), most likely (M), and pessimistic (P) estimates to represent uncertainty. A common PERT expected-value formula is (O + 4M + P) / 6.

Teams often combine methods—for example, bottom-up estimates for the WBS, analogy for a new feature, and a model-based estimate as a reasonableness check.

4. COCOMO: Constructive Cost Model

COCOMO is a family of software cost-estimation models developed to estimate effort and schedule from software size and other project characteristics. The original Basic COCOMO model uses size measured in thousands of delivered source lines of code (KLOC) and classifies projects into development modes.

The classic modes are:

Mode	General description	Typical characteristics
Organic	Relatively familiar, smaller projects	Experienced team, flexible environment, comparatively few constraints
Semi-detached	Intermediate complexity	Mixed experience and constraints
Embedded	Strong constraints or tight coupling with hardware, regulations, or operational requirements	High constraint levels and complex interactions

These labels come from the original COCOMO model. Real projects do not always fit neatly into one mode, and modern estimation may use other models, measures, and historical data.

5. Basic COCOMO Equations

For Basic COCOMO, the classic equations are:

Effort (person-months) = a × (KLOC)^{b}
Development time (months) = c × (Effort)^{d}

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Use KLOC, not raw lines of code, in these equations. The results are model estimates, not exact predictions. The basic model does not explicitly include all modern factors such as reuse, cloud services, team distribution, or detailed product attributes.

6. Worked Basic COCOMO Example

Suppose a small, relatively familiar CEMS project is provisionally classified as organic and estimated at 10 KLOC. This is an illustrative assumption for calculation practice.

Step 1 — Effort
Effort = 2.4 × $(10)^{1.05}$
$10^{1.05} \approx 11.22$

Effort ≈ 2.4 × 11.22 = **26.9 person-months**.

Step 2 — Development time
Time = $2.5 \times (26.9)^{0.38}$
Time ≈ **9.1 months** (approximate).

Step 3 — Interpret the result
The estimate suggests the scale of effort and duration under the model's assumptions. It does not mean exactly three people can finish in nine months; staffing and task dependencies must be planned separately. Validate the size and project mode before using the result for a real commitment.

7. From Effort to a Budget

A simple labour-cost estimate can be calculated as:

Labour cost = estimated effort × cost per person-month

If the estimated effort is 26.9 person-months and the fully loaded cost is assumed to be ■1,00,000 per person-month, labour cost is approximately ■26,90,000. Add applicable hosting, licences, training, travel, contingency, and other costs separately. The rate here is only an example, not a market benchmark.

Avoid adding a contingency percentage without explaining the risks it covers. A reserve should reflect uncertainty and the organization's estimating policy.

8. Risk Management Fundamentals

A project risk is an uncertain event or condition that, if it occurs, can affect project objectives. A risk is not the same as an issue: a risk may happen in the future; an issue is a problem that has already occurred.

Term	Meaning	CEMS example
Risk	Uncertain future event or condition	The email provider may limit message volume during registration opening
Issue	A current problem that needs action	The email provider has disabled the project's credentials
Probability	How likely the risk is to occur	Low, medium, or high; or a defined numeric estimate
Impact	How strongly it could affect objectives	Minor inconvenience or blocked event registration
Mitigation	Action to reduce probability or impact	Load-test notifications and define retry behavior
Contingency	Planned action if the risk occurs	Use an approved fallback communication process

9. Risk Management Process

1. Identify risks from requirements, technology, team capacity, suppliers, security, and schedule dependencies.
2. Describe each risk clearly as cause → uncertain event → effect.
3. Assess probability and impact using agreed scales.
4. Prioritize risks; consider urgency and how difficult they would be to detect or recover from.
5. Choose a response: avoid, mitigate, transfer/share where appropriate, or accept with a plan.
6. Assign a risk owner and target actions or review date.
7. Monitor risks during the project and update the register as conditions change.

Example risk statement: “Because the event notification provider has a rate limit, confirmation emails may be delayed during a registration spike, causing students to be unsure whether their registrations succeeded.”

10. Sample CEMS Risk Register

ID / risk	Probability	Impact	Response / owner
R1: Requirements change close to release	Medium	High	Confirm scope and use change control; project lead
R2: Registration service fails under peak load	Medium	High	Load test, monitor, and prepare capacity plan; technical lead
R3: Unauthorized access to organizer functions	Low–Medium	Very high	Review authorization rules and test role boundaries; security owner
R4: Key team member unavailable	Medium	Medium	Document work, pair on critical areas, and cross-train; team lead
R5: Notification provider is unavailable	Medium	Medium	Retry safely and provide clear in-app confirmation; integration owner

Risk scores should be interpreted with care. A low-probability security risk can still deserve high priority when the impact is severe. Use the team's agreed risk scale rather than treating a simple score as an automatic decision.

11. Classroom Activity: Estimate and Manage Risk

1. Choose a small software project and define its scope and size assumption.
2. Estimate effort using a suitable method; if using Basic COCOMO, state the mode and KLOC estimate.
3. Convert effort to a tentative cost using an explicitly stated person-month rate.
4. Identify at least five risks and record cause, event, effect, probability, impact, owner, and response.
5. Choose the two highest-priority risks and explain why they matter most.
6. State which estimates are uncertain and what evidence would improve them.

> **Deliverable:** a one-page estimation sheet and a risk register with owners and planned responses.

12. Review Questions with Suggested Answers

1. How does effort differ from duration?

Suggested answer: Effort measures person-work; duration measures elapsed time.

2. Name three estimation approaches.

Suggested answer: Expert judgment, analogy, bottom-up decomposition, parametric models, and three-point estimation are examples.

3. What does KLOC mean?

Suggested answer: Thousands of lines of source code.

4. Write the Basic COCOMO effort equation.

Suggested answer: Effort = a × (KLOC)^b, where a and b depend on the development mode.

5. What are the three classic Basic COCOMO modes?

Suggested answer: Organic, semi-detached, and embedded.

6. Why is a COCOMO result not an exact commitment?

Suggested answer: It depends on size estimates, mode classification, model assumptions, and factors not fully represented by the basic model.

7. Distinguish a risk from an issue.

Suggested answer: A risk is uncertain and may occur; an issue is a current problem.

8. What is mitigation?

Suggested answer: An action that reduces the probability or impact of a risk.

9. Why assign a risk owner?

Suggested answer: To make monitoring and response actions accountable.

10. Why might a low-probability risk still be important?

Suggested answer: Its impact may be severe, such as unauthorized access or loss of critical data.

13. Glossary

Term	Meaning
Effort	Amount of person-work required
Duration	Elapsed time from start to finish
KLOC	Thousands of lines of source code
COCOMO	A family of software cost-estimation models
Organic mode	Classic COCOMO category for relatively familiar, less constrained projects
Risk	Uncertain event or condition that may affect objectives
Issue	A problem that has already occurred
Mitigation	Action intended to reduce a risk's probability or impact
Contingency plan	Prepared response if a risk occurs
Risk owner	Person accountable for monitoring a risk and its response

14. Homework

1. Estimate effort and development time for a 5 KLOC organic project using the Basic COCOMO coefficients in this chapter. Show your formula and working.

2. Explain why adding people does not always reduce the project duration proportionally.

3. Prepare a risk register for a Library Management System with at least six risks.

4. For each risk, state a cause, event, effect, probability, impact, owner, and response.

5. Identify three assumptions in a software estimate and explain how you would validate them.

Next chapter: Configuration Management, Version Control with Git, and Change Management.